Seat Map Feature

NDC v2 seat selection — technical documentation

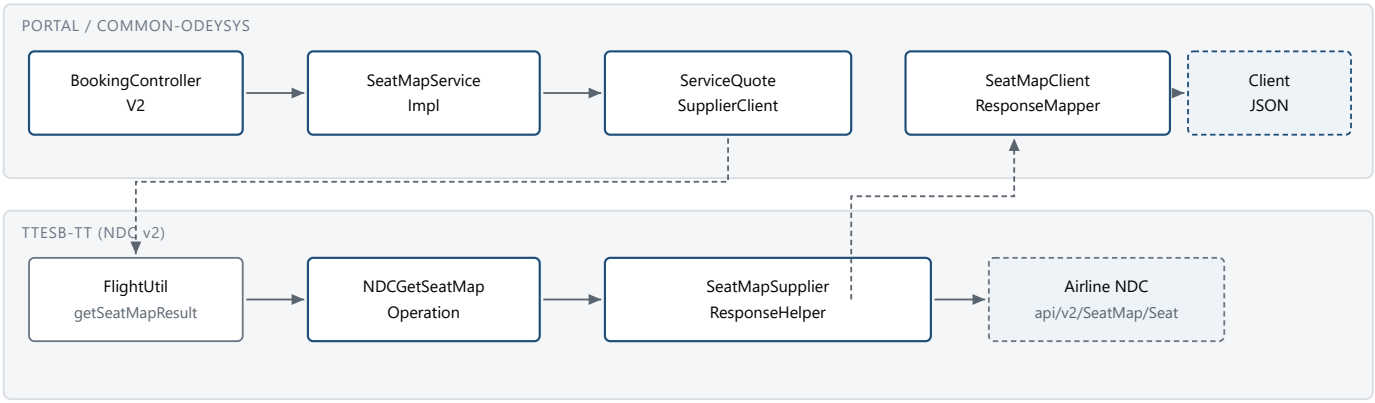
Odeysys · modules `common-odeysys` , `ttesb-tt` , `odeysysportal-tt` , `odeysysadmin-tt`

1. Overview

Retrieves an airline seat map for a held offer, renders it as a per-passenger grid, and carries the passenger's seat selection through to booking. Built on the NDC v2 integration layer, so it works for any v2 supplier rather than one carrier.

Aspect	Detail
Endpoint	`POST .../v2/booking/seat-map` SEAT_MAP
Request	`SsrServiceQuoteRequest` — offer id, response id, passenger list
Response	`RetrieveSeatMapResponseForClient`
Supplier call	`POST api/v2/SeatMap/Seat`
Verified against	FlyNas, RiyadhAir
Service key	`ProductEnum.Flight.GET_SEAT_MAP`

2. Request flow



1. **Build request** — `SeatMapServiceImpl` loads the cached flight option via `PaxServiceUtil` and assembles `RetrieveServiceQuotesRequestBean` .
2. **Dispatch** — `ServiceQuoteSupplierClient` resolves the supplier connector through the ESB service factory.
3. **Supplier call** — `NDCGetSeatMapOperation` posts the offer id and selected bundles, receives `RetrieveSeatMapResponseFromSupplier` .
4. **Normalise** — `SeatMapSupplierResponseHelper` matches passengers, converts prices, flattens the cabin into seat matrices.
5. **Shape for client** — `SeatMapClientResponseMapper` builds the deck → zone → row → cell tree and signs each per-passenger offer.
6. **Snapshot** — quoted prices are written to the HTTP session for the booking-time tamper check.

3. Main classes

common-odeysys — application layer

Class	Responsibility
`SeatMapService`	Interface — single entry point `fetchSeatMap`.
`SeatMapServiceImpl`	Orchestrates the call: builds the request bean, invokes the supplier, refreshes the cached flight option, snapshots prices into session.
`SeatMapClientResponseMapper`	Converts supplier seat matrices into the client contract. Builds the pax-eligibility index, resolves currency, signs every seat offer.
`ServiceQuoteSupplierClient`	Thin wrapper over `ServiceResolverFactory`; raises `BadRequestExc` on a supplier error message.
`SsrServiceQuoteHelper`	Writes the refreshed offer / response ids back into the Redis-cached flight option.
`PaxServiceUtil`	Shared loader that populates `FlightBookingBean` from the cached one-way / multi-city / round-trip option.

ttesb-tt — integration layer

Class	Responsibility
`NDCGetSeatMapOperation`	NDC v2 operation. Builds the supplier request (offer id, bundles, credentials), resolves the exchange rate, delegates mapping.
`SeatMapSupplierResponseHelper`	Core translation. Matches supplier passengers to ours by identity, re-keys prices to the client pax id, converts currency, flattens rows by deck.
`SeatMapBookRequestBuilder`	Collects each passenger's chosen seat per segment for the book / book-and-hold request.
`FlightUtil`	ESB dispatch — routes `GET_SEAT_MAP` to the supplier connector.

Key DTOs

Type	Notes
`RetrieveSeatMapResponseForClient`	Root response — currency, passengers, segments, offer / response id.
`SeatSegment`	One flight leg with its carrier / timing metadata and cabin layout.
`CabinLayout` → `Deck` → `Zone`	Cabin structure. `DeckType`: MAIN / UPPER / LOWER / UNKNOWN.
`ZoneRow` → `RowCell`	A physical row and its cells. `ColumnType`: SEAT / AISLE / LAVATORY.
`PassengerSeatOfferDto`	Per-passenger offer on one cell: eligibility, price, signature, expiry.
`SeatResponsePaxDto`	Passenger identity echoed back with its client-facing id.

4. Response structure

```
RetrieveSeatMapResponseForClient
├── currency, offerId, responseId
├── passengers[]  →  SeatResponsePaxDto { passengerId, type, name }
└── segments[]    →  SeatSegment
                     ├── segmentId, origin, destination
                     ├── departure/arrivalDateTime, terminals
                     ├── operating/marketingCarrierCode, flightNumber
                     ├── flightTime, equipment
                     └── cabinLayout
                         └── decks[]  →  Deck { deck: MAIN | UPPER }
                             └── zones[]  →  Zone { zoneId, cabinClass, rowCount, columnCount }
                                 ├── columns[]  →  ZoneColumn { columnId, label, zoneType }
                                 └── rows[]     →  ZoneRow { rowNumber }
                                     └── cells[]  →  RowCell
                                         ├── cellId, cellCode, column
                                         ├── cellCategory, cellAvailability
                                         ├── cellCharacteristics[]
                                         └── passengerSeatOffers[]  →  PassengerSeatOfferDto
                                                                       { passengerId, eligible,
                                                                         price, signature, expiresAt }
```

5. Design decisions

5.1 Supplier-agnostic passenger matching

Each supplier keys its passenger block differently — FlyNas sends a zero-based index (`"0"` , `"1"`), RiyadhAir a traveller reference (`"T1"` , `"T2"`). The key is therefore never parsed. `mapAvailableSeatPerPax` matches each supplier entry to one of our passengers on the identity the supplier echoes back.

Field	Weight	Behaviour
documentNumber	4	Passport / document id
birthDate	3	Date part only — supplier appends a time component
firstName / lastName	2 each	Trimmed, case-insensitive
middleName	1	Scores when both agree; never disqualifies (routinely blank)
passengerTypeCode	1	ADT / CHD / INF

- **A conflict disqualifies.** If a field is populated on both sides and differs, that passenger is excluded outright — no score can override it.
- **A blank field is neutral.** Missing data contributes zero rather than rejecting the match.
- **One-to-one.** Each passenger is claimed at most once.
- **Fallback.** A supplier that echoes no identity at all still resolves through the legacy zero-based key.

5.2 A single passenger-id space

The client contract uses a zero-based id (`paxSequence - 1`). Supplier seat prices arrive under the supplier's own key. `mapAvailableSeatPerPax` returns the `supplier key → client pax id` translation, and `getPriceAfterConversion` applies it as prices are copied — the one place the supplier id space ends.

> **Why it matters:** `buildPassengerOffers` looks up eligibility and price with the same id. Without the translation, a RiyadhAir price map keyed `"T1"` would silently miss every `get("0")` and quote every seat at zero.

5.3 Tamper protection

Two independent mechanisms guard the quoted price:

- **Signature** — every `PassengerSeatOfferDto` is signed by `DataSignatureService` over eligibility + price, with an expiry, so a selected seat can be echoed back verbatim through fare check and booking.
- **Session snapshot** — `addSeatMapInSession` flattens every quoted price into the session under `_SSRSeat` , keyed by passenger + amount + seat + zone + seat id, for re-verification at booking time.

5.4 Currency and conversion

The supplier states its currency only on the seat prices, so `getCurrency` takes it from the first priced cell — preferring per-passenger prices, falling back to the flat price. Amounts are converted with the rate from `ExchangeRateFactory` ; `SeatMapClientResponseMapper.resolveCurrency` then reads the currency off the response rather than the request configuration.

6. Known limitations

Item	Detail
Return leg not quoted	`addSeatMapInSession` reads the onward leg only; `_SSRSeatr` is written empty.

Multi-deck eligibility	Eligibility is stored per segment with `put`, so on a two-deck aircraft the second deck overwrites the first. Not reachable on single-deck fleets.
Missing exchange rate	The rate defaults to `1.0`, leaving amounts in the supplier's currency without signalling it.
Identical passengers	Two passengers with the same name and no document / birth date resolve by list order.
Seat cost in SSR total	`AncillaryHelper.getTotalSSRPrice` sums baggage and meal only; seat cost is not included in the v2 ancillary total.